

ASTANA POLYTECHNIC

[Бөлім / Кафедра атауы]

ДИПЛОМДЫҚ ЖОБА

Әдістемелік (теориялық) бөлім

Тақырыбы: «SmartHub робототехника және дрон академиясының веб-сайтын әзірлеу»

Орындаған: [Аты-жөні] _____

Тобы: [Тобы]

Жетекшісі: [Жетекшінің аты-жөні] _____

Нормобақылаушы: [Аты-жөні] _____

Рецензент: [Аты-жөні] _____

Шифр: [ДЖ.xxxxxxxx.W-xx-xx]

Астана — 2026

МАЗМҰНЫ

КІРІСПЕ

1. ЖАЛПЫ БӨЛІМ

- 1.1 Жобаның жалпы сипаттамасы
- 1.2 Заманауи веб-технологияларға шолу
- 1.3 Ұқсас шешімдерге талдау

2. ТЕХНОЛОГИЯЛЫҚ БӨЛІМ

- 2.1 Жобаға қойылатын талаптар
- 2.2 Жобаның жалпы архитектурасы
- 2.3 Қолданылған құралдар мен технологиялар
- 2.4 Жобаның файлдық құрылымы
- 2.5 Дизайн және түстік схема

3. БАҒДАРЛАМАЛЫҚ ІСКЕ АСЫРУ — КЛИЕНТТІК БӨЛІК

- 3.1 HTML — беттің семантикалық құрылымы
- 3.2 CSS — стильдеу және бейімделгіш дизайн
- 3.3 JavaScript — интерактивтік логика

4. СЕРВЕРЛІК БӨЛІК

- 4.1 Node.js және Express негіздері
- 4.2 Деректер базасы (submissions.json)
- 4.3 API эндпоинттері
- 4.4 Excel форматына экспорт

5. АДМИНИСТРАТОРЛЫҚ ПАНЕЛЬ

- 5.1 Аутентификация (кіру кілті)
- 5.2 Заявқаларды басқару

6. ҚАУІПСІЗДІК, ТЕСТІЛЕУ ЖӘНЕ ОРНАЛАСТЫРУ

ҚОРЫТЫНДЫ

ПАЙДАЛАНЫЛҒАН ӘДЕБИЕТТЕР

КІРІСПЕ

Қазіргі ХХІ ғасыр — ақпараттық технологиялардың қарқынды дамыған дәуірі. Кез келген білім беру мекемесінің, кәсіпорынның немесе ұйымның интернет желісіндегі өкілдігі, яғни веб-сайты, бүгінгі таңда тек беделдің көрсеткіші ғана емес, сонымен қатар клиенттермен байланыс орнатудың, қызметтерді таныстырудың және өтінімдерді қабылдаудың негізгі құралына айналды. Әсіресе білім беру саласында сапалы әрі ыңғайлы веб-ресурс ата-аналар мен оқушылардың мекеме туралы алғашқы әсерін қалыптастырады.

Бұл дипломдық жобаның тақырыбы — «SmartHub робототехника және дрон академиясының веб-сайтын әзірлеу». SmartHub — Қазақстандағы алғашқы робототехника және дрон академиясы, ол заманауи технологияларды, инженерлік білім беруді және практикалық дағдыларды біріктіреді. Академияның мақсаты — инженерлердің, әзірлеушілердің, дрон операторларының және технологиялық кәсіпкерлердің жаңа буынын дайындау. Осы мақсаттарға жету үшін академияға өзінің қызметтерін, жетістіктерін және оқу бағдарламаларын кеңінен таныстыратын заманауи веб-сайт қажет болды.

Жобаның өзектілігі бірнеше факторлармен айқындалады. Біріншіден, білім беру нарығындағы бәсекелестік жағдайында академияның онлайн-өкілдігі әлеуетті клиенттерді тартудың тиімді тәсілі болып табылады. Екіншіден, сайт арқылы ата-аналар пробалық сабаққа онлайн тіркеле алады, бұл өтінімдерді қабылдау процесін айтарлықтай жеңілдетеді. Үшіншіден, барлық деректерді бір жерде сақтайтын әрі оларды Excel форматына экспорттай алатын әкімшілік панель академия қызметкерлерінің жұмысын автоматтандырады.

Жобаның мақсаты — SmartHub академиясы үшін заманауи, бейімделгіш (адаптивті) және интерактивті веб-сайтты, оған қоса өтінімдерді қабылдайтын серверлік бөлікті және оларды басқаратын әкімшілік панельді әзірлеу.

Қойылған мақсатқа жету үшін келесі міндеттер шешілді:

- пәндік аймақты талдау және сайтқа қойылатын талаптарды анықтау;

- заманауи веб-технологияларды (HTML5, CSS3, JavaScript, Node.js) зерттеу және таңдау;
- сайттың құрылымы мен дизайнын жобалау;
- беттің семантикалық HTML-құрылымын құру;
- CSS көмегімен бейімделгіш дизайн және күндізгі/түнгі тақырыпты іске асыру;
- JavaScript арқылы интерактивтік элементтерді (аударылатын карточкалар, отзывтар каруселі, лайтбокс, тақырып ауыстырғыш) бағдарламалау;
- Express негізіндегі серверді және өтінімдерді сақтайтын деректер базасын құру;
- деректерді Excel форматына экспорттау функциясын іске асыру;
- өтінімдерді қарауға, жоюға және экспорттауға арналған әкімшілік панельді әзірлеу;
- жобаны тестілеу және орналастыруға дайындау.

Зерттеу нысаны — білім беру мекемесінің веб-өкілдігін құру процесі. Зерттеу пәні — SmartHub академиясының веб-сайтын әзірлеуге арналған технологиялар мен әдістер.

Жобаның практикалық маңыздылығы — әзірленген веб-сайт нақты академияның күнделікті жұмысында қолданыла алады: ол жаңа клиенттерді тартады, өтінімдерді автоматты түрде жинайды және академияның онлайн-беделін арттырады.

Дипломдық жобаның бұл әдістемелік бөлімінде жобаның теориялық негіздері, қолданылған технологиялар, бағдарламалық кодтың құрылымы мен жұмыс істеу принципі, серверлік бөлік, деректер базасы және әкімшілік панель толық сипатталады. Әр тарауда нақты бағдарламалық код фрагменттері келтіріліп, олардың жұмысы егжей-тегжейлі түсіндіріледі.

1. ЖАЛПЫ БӨЛІМ

1.1 Жобаның жалпы сипаттамасы

SmartHub академиясының веб-сайты — бұл бір беттік (one-page) құрылымдағы заманауи корпоративтік сайт. Бір беттік құрылым пайдаланушыға беттер арасында ауыспай-ақ, тек төмен айналдыру (скролл) арқылы академия туралы барлық негізгі ақпаратпен танысуға мүмкіндік береді. Бұл тәсіл мобильді құрылғыларда әсіресе ыңғайлы және қазіргі заманғы лендинг-сайттарға тән.

Сайт логикалық тұрғыдан бірнеше мағыналық бөлімге (секцияға) бөлінген:

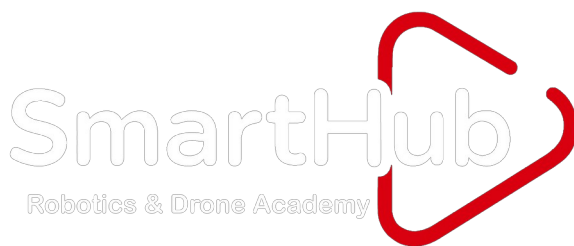
- **Басты экран (Hero)** — академияның атауы, ұраны және «О нас» батырмасы бар бірінші экран. Фонда академия командасының халықаралық жарыстағы суреті орналасқан.
- **Біз туралы** — академияның миссиясы мен мақсатын ашатын, тінтуірмен немесе тиекпен аударылатын интерактивтік карточкалар.
- **Оқу бағдарламасы** — Робототехника бағытының деңгейлері (Junior, Middle, Master, Expert) және Дрондар, Дрон-футбол, Спорттық РВТ бағыттары.
- **Біз қай елдерде болдық** — академия қатысқан елдердің тулары; жетістігі бар тулар аударылып, нақты жеңісті көрсетеді.
- **Жетістіктер** — жыл қорытындысы (сандармен) және халықаралық жарыстардан түсірілген нақты суреттерден тұратын фотогалерея.
- **Пікірлер** — ата-аналардың пікірлерін көрсететін, оқтармен және нүктелермен парақталатын карусель.
- **Байланыс** — академияның филиалдары, байланыс деректері және пробалық сабаққа тіркелуге арналған өтінім формасы.
- **Төменгі бөлім (Footer)** — қысқаша ақпарат пен авторлық құқық.

Сайт екі негізгі бөліктен тұрады: **клиенттік бөлік** (frontend) — пайдаланушы браузерде көретін бет, және **серверлік бөлік** (backend) —

өтінімдерді қабылдап, сақтап, әкімшіге беретін бағдарлама. Бұған қоса жеке **әкімшілік панель** бар, ол арқылы академия қызметкері барлық түскен өтінімдерді көріп, жойып немесе Excel файлына түсіріп ала алады.

1.1-кесте. Жобаның негізгі сипаттамалары

Көрсеткіш	Мәні
Жоба түрі	Бір беттік корпоративтік веб-сайт + серверлік бөлік
Клиенттік технологиялар	HTML5, CSS3, JavaScript (ES6+)
Серверлік технологиялар	Node.js, Express.js
Деректер базасы	Файлдық JSON-база (submissions.json)
Қосымша кітапханалар	xlsx (Excel экспорты)
Сыртқы ресурстар	Google Fonts, flagcdn.com (тулар)
Бейімделгіштік	Компьютер, планшет, смартфон
Тақырып режимі	Күндізгі (light) және түнгі (dark)



1.1-сурет. SmartHub академиясының логотипі

1.2 Заманауи веб-технологияларға шолу

Веб-сайт әзірлеу үшін бүгінгі күні бірнеше негізгі технология қолданылады. Олардың әрқайсысы белгілі бір рөл атқарады және өзара үйлесімде толыққанды өнім жасауға мүмкіндік береді. Төменде осы жобада қолданылған технологиялардың теориялық сипаттамасы келтірілген.

1.2.1 HTML — гипермәтіндік белгілеу тілі

HTML (HyperText Markup Language) — веб-беттің мазмұны мен құрылымын анықтайтын белгілеу тілі. HTML тілі тегтер (tags) деп аталатын арнайы белгілер арқылы беттің элементтерін — тақырыптарды, абзацтарды, суреттерді, сілтемелерді, формаларды — сипаттайды. Бұл жобада HTML5 нұсқасы қолданылған, оның басты артықшылығы — семантикалық тегтердің (section, header, footer, nav, figure) болуы. Семантикалық тегтер беттің құрылымын тек визуалды ғана емес, мағыналық тұрғыдан да анық етеді, бұл іздеу жүйелерінің сайтты дұрыс индекстеуіне (SEO) және қолжетімділікке оң әсер етеді.

1.2.2 CSS — каскадтық стиль кестелері

CSS (Cascading Style Sheets) — HTML элементтерінің сыртқы көрінісін, яғни түсін, өлшемін, орналасуын, шрифтін және анимациясын анықтайтын тіл. HTML беттің «қаңқасын» сипаттаса, CSS оған «киім кигізеді». Бұл жобада CSS3-тің заманауи мүмкіндіктері кеңінен қолданылды:

- **CSS айнымалылары** — түстерді бір жерден басқару және күндізгі/түнгі тақырыпты ауыстыру үшін;
- **Flexbox және Grid** — элементтерді икемді әрі бейімделгіш түрде орналастыру үшін;
- **3D-түрлендірулер (transform, perspective)** — карточкаларды аудару әсері үшін;
- **Медиа-сұраулар (media queries)** — экранның еніне қарай бетті бейімдеу үшін;
- **Өтпелер мен анимациялар (transition)** — тегіс визуалды әсерлер үшін.

1.2.3 JavaScript — бағдарламалау тілі

JavaScript — веб-беттерге интерактивтілік қосатын бағдарламалау тілі. HTML мен CSS статикалық көрініс берсе, JavaScript пайдаланушының әрекеттеріне (батырманы басу, тінтуірді апару, форманы толтыру) жауап беретін динамикалық мінез-құлықты қамтамасыз етеді. Бұл жобада JavaScript тілінде келесі функциялар іске асырылды: тақырыпты ауыстыру, навигацияның

белсенді сілтемесін бөлектеу, аударылатын карточкалар, пікірлер каруселі, фото лайтбоксы және өтінім формасын өңдеу.

1.2.4 Node.js және Express

Node.js — JavaScript тілін браузерден тыс, серверде орындауға мүмкіндік беретін орта (runtime). Node.js арқасында бір тілде (JavaScript) әрі клиенттік, әрі серверлік бөлікті жазуға болады, бұл әзірлеуді жеңілдетеді. Express — Node.js үшін жеңіл әрі танымал веб-фреймворк, ол HTTP-сұраныстарды өңдеуді, маршруттауды (routing) және статикалық файлдарды беруді айтарлықтай оңайлатады. Осы жобада сервер Express негізінде құрылған.

1.2.5 HTTP протоколы және JSON форматы

Клиент пен сервер арасындағы байланыс HTTP (HyperText Transfer Protocol) протоколы арқылы жүзеге асырылады. HTTP — өтінім-жауап (request-response) принципінде жұмыс істейтін протокол: клиент серверге сұраныс (request) жібереді, сервер оны өңдеп, жауап (response) қайтарады. Әр сұраныстың әдісі (method) болады, олардың ішінде ең жиі қолданылатындары: GET (деректерді алу), POST (жаңа дерек жіберу), DELETE (дерек жою). Бұл жобада осы үш әдіс те қолданылады.

Клиент пен сервер арасында деректер JSON (JavaScript Object Notation) форматында алмасылады. JSON — адам оқи алатын, әрі бағдарлама оңай өңдейтін мәтіндік формат. Ол «кілт: мән» жұптарынан тұрады және деректерді құрылымдап ұсынуға ыңғайлы. Бұл жобада өтінімдер де, деректер базасы да JSON форматында сақталады.

1.3 Ұқсас шешімдерге талдау

Жобаны әзірлемес бұрын ұқсас білім беру мекемелерінің сайттарына талдау жасалды. Талдау нәтижесінде осы саладағы сайттарға тән бірнеше ортақ белгі анықталды: бірінші экранда мекеменің атауы мен ұраны, оқу бағыттарының тізімі, жетістіктер бөлімі, ата-аналар пікірлері және байланыс формасы. Алайда көптеген сайттарда мобильді нұсқаның нашар бейімделуі, түнгі режимнің болмауы және өтінімдерді қолмен өңдеу мәселелері кездесті.

Осы кемшіліктерді ескере отырып, SmartHub сайтында келесі артықшылықтарды іске асыру шешілді: толық бейімделгіш дизайн, күндізгі/түнгі тақырып, интерактивтік аударылатын карточкалар мен фотогалерея, сондай-ақ өтінімдерді автоматты түрде жинап, Excel-ге экспорттай алатын әкімшілік панель. Бұл шешімдер сайтты бәсекелестерден ерекшелендіреді.

2. ТЕХНОЛОГИЯЛЫҚ БӨЛІМ

2.1 Жобаға қойылатын талаптар

Веб-сайтты әзірлеу алдында оған қойылатын функционалдық және функционалдық емес талаптар анықталды.

Функционалдық талаптар — сайт нені орындауы керек:

- академия туралы толық ақпаратты (миссия, бағыттар, жетістіктер) ұсыну;
- пробалық сабаққа тіркелуге арналған өтінім формасын ұсыну;
- өтінім деректерін серверге жіберіп, сақтау;
- интернет болмаған жағдайда деректерді браузердің жадында (localStorage) сақтау;
- әкімшіге барлық өтінімдерді көру, жою және Excel-ге экспорттау мүмкіндігін беру;
- оқу бағыттары мен жетістіктерді интерактивтік түрде көрсету.

Функционалдық емес талаптар — сайт қалай жұмыс істеуі керек:

- **Бейімделгіштік** — сайт компьютерде, планшетте және смартфонда бірдей дұрыс көрінуі тиіс;
- **Жылдамдық** — беттің тез жүктелуі (суреттерге lazy-loading қолдану);
- **Ыңғайлылық (UX)** — навигацияның қарапайымдылығы, түсінікті интерфейс;
- **Қолжетімділік** — семантикалық тегтер мен aria-атрибуттарын қолдану;
- **Сенімділік** — сервер қолжетімсіз болса да, өтінімнің жоғалмауы;
- **Қауіпсіздік** — әкімшілік панельге кілтпен қол жеткізу.

2.2 Жобаның жалпы архитектурасы

Жоба «клиент—сервер» архитектурасы бойынша құрылған. Бұл архитектурада екі тарап өзара әрекеттеседі: **клиент** (пайдаланушының браузері) серверге сұраныс жібереді, ал **сервер** сұранысты өңдеп, жауап қайтарады. Жобаның жалпы жұмыс схемасы келесідей:

1. Пайдаланушы браузерде сайтты ашады — Express-сервер статикалық файлдарды (index.html, style.css, script.js, суреттер) береді.
2. Пайдаланушы өтінім формасын толтырып жібереді — браузердегі JavaScript деректерді алдымен localStorage-қа сақтайды, содан кейін серверге POST-сұраныс арқылы жібереді.
3. Сервер деректерді қабылдап, submissions.json файлына (деректер базасына) жазады.
4. Әкімші admin.html бетін ашып, кілтті енгізеді — сервер барлық өтінімдерді қайтарады.
5. Әкімші қажет болса өтінімді жояды немесе барлығын Excel файлына экспорттайды.

Мұндай бөлінудің артықшылығы — клиенттік және серверлік бөліктер бір-бірінен тәуелсіз дамытылып, тестіленуі мүмкін. Сонымен қатар деректерді сақтау логикасы серверде шоғырланғандықтан, қауіпсіздік пен тұтастық жоғары болады.

2.3 Қолданылған құралдар мен технологиялар

Жобаны әзірлеу барысында келесі құралдар мен технологиялар пайдаланылды (2.1-кесте).

2.1-кесте. Қолданылған құралдар

Құрал / технология	Тағайындалуы
HTML5	Беттің құрылымы мен мазмұны
CSS3	Стильдеу, анимация, бейімделгіш дизайн
JavaScript (ES6+)	Клиенттік интерактивтік логика
Node.js	Серверлік ортаны қамтамасыз ету
Express.js	Веб-сервер және маршруттау
xlsx	Деректерді Excel форматына түрлендіру
JSON	Деректерді сақтау форматы

Visual Studio Code	Код жазуға арналған редактор
Git	Нұсқаларды бақылау жүйесі
Google Fonts	Playfair Display және Inter қаріптері

Жобаның тәуелділіктері (dependencies) package.json файлында сипатталған. Бұл файл жобаның атауын, нұсқасын, іске қосу командаларын және қажетті кітапханаларды қамтиды:

```
{
  "name": "smarthub-academy",
  "version": "1.0.0",
  "description": "SmartHub Robotics & Drone Academy",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "dev": "node server.js"
  },
  "dependencies": {
    "express": "^4.18.3",
    "xlsx": "^0.18.5"
  }
}
```

Мұндағы scripts бөлімі серверді «npm start» командасымен іске қосуға мүмкіндік береді, ал dependencies бөлімінде жобаға қажетті екі сыртқы кітапхана — express және xlsx — көрсетілген. Бұл кітапханалар «npm install» командасы арқылы автоматты түрде орнатылады.

2.4 Жобаның файлдық құрылымы

Жоба файлдары логикалық түрде ұйымдастырылған. Әр файл белгілі бір рөл атқарады (2.2-кесте).

2.2-кесте. Жобаның негізгі файлдары

Файл	Тағайындалуы
index.html	Сайттың басты беті (барлық секциялар)
style.css	Сайттың барлық стильдері
script.js	Клиенттік интерактивтік логика
server.js	Express-сервер және API
admin.html	Әкімшілік панель

submissions.json	Өтінімдер сақталатын деректер базасы
package.json	Жоба сипаттамасы және тәуелділіктер
hero-bg.jpeg, ach1–ach6	Сайт суреттері (фон, галерея)
node_modules/	Орнатылған кітапханалар

2.5 Дизайн және түстік схема

Сайттың дизайны академияның фирмалық стиліне сай жасалды. Негізгі түс — қызыл (#CC0000), ол академияның логотипі мен бренд түсіне сәйкес келеді. Қосымша түс ретінде көк (#1A1A8C) және қара (#1A1A1A) қолданылды. Қаріптер ретінде тақырыптар үшін кербез Playfair Display, ал негізгі мәтін үшін оқуға ыңғайлы Inter қаріптері таңдалды.

Барлық түстер CSS айнымалылары арқылы бір жерде анықталған, бұл оларды басқаруды және түнгі тақырыпқа ауыстыруды жеңілдетеді:

```
:root {
  --red: #CC0000;
  --blue: #1A1A8C;
  --dark: #1a1a1a;
  --light-bg: #F0EFEE;
  --white: #ffffff;
  --card-bg: #ffffff;
  --nav-bg: #ffffff;
}
```

Жоғарыдағы код фрагментінде :root селекторы құжаттың түбірлік элементін білдіреді, ал ішіндегі «--» префиксі бар жазбалар — CSS айнымалылары. Мысалы, --red айнымалысы қызыл түсті сақтайды және кодтың кез келген жерінде var(--red) деп шақырылады. Бұл тәсілдің артықшылығы: егер бренд түсін өзгерту керек болса, тек бір жерде ғана түзету жеткілікті.

2.6 Жобаның жұмыс схемасы

Жобаның жалпы жұмыс циклін деректердің қозғалу схемасы (data flow) арқылы сипаттауға болады. Пайдаланушы өтінім бергеннен бастап, ол әкімшіге жеткенге дейінгі деректер жолы келесідей кезеңдерден өтеді:

6. Пайдаланушы браузерде форманы толтырып, «Записаться» батырмасын басады.

7. Браузердегі script.js деректерді жинап, алдымен localStorage-қа (браузер жадына) жазады — бұл деректің жоғалмауының кепілі.
8. Содан кейін деректер fetch арқылы JSON форматында серверге (POST /api/submit) жіберіледі.
9. Express-сервер деректерді тексеріп (валидация), submissions.json деректер базасына жазады.
10. Академия қызметкері admin.html панелін ашып, кілтпен кіреді.
11. Панель серверден (GET /api/submissions) барлық өтінімдерді сұрап алады және кесте түрінде көрсетеді.
12. Қажет болса, қызметкер өтінімді жояды (DELETE) немесе барлығын Excel файлына (GET /api/export) экспорттайды.

Бұл схема жобаның екі негізгі ерекшелігін көрсетеді: біріншіден, деректер екі жерде (браузерде және серверде) сақталатындықтан сенімділік жоғары; екіншіден, әкімшілік панель серверлік API арқылы деректер базасымен байланысып, қызметкерге ыңғайлы интерфейс ұсынады.

3. БАҒДАРЛАМАЛЫҚ ІСКЕ АСЫРУ — КЛИЕНТТІК БӨЛІК

3.1 HTML — беттің семантикалық құрылымы

Сайттың барлық мазмұны index.html файлында орналасқан. Беттің жалпы қаңқасы стандартты HTML5 құрылымына сәйкес: тілді көрсететін html тегі, метадеректер мен сыртқы файлдарды қосатын head бөлімі және көрінетін мазмұнды қамтитын body бөлімі.

```
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>SmartHub – Robotics & Drone Academy</title>
  <link href="https://fonts.googleapis.com/css2?family=Playfair+Display..."
rel="stylesheet" />
  <link rel="stylesheet" href="style.css" />
</head>
<body>
  ... барлық секциялар ...
  <script src="script.js?v=5"></script>
</body>
</html>
```

Мұндағы <meta charset="UTF-8"> тегі беттің кодтауын белгілейді (кириллица мен қазақ әріптерінің дұрыс көрінуі үшін), ал viewport мета-тегі мобильді құрылғыларда беттің дұрыс масштабталуын қамтамасыз етеді. <link> тегтері арқылы Google Fonts қаріптері мен style.css стиль файлы қосылады. script.js файлы беттің соңында қосылады — бұл барлық HTML элементтері жүктелгеннен кейін ғана скриптің іске қосылуын қамтамасыз етеді. Файл атауындағы «?v=5» — кэшті жаңарту тәсілі: нұсқа нөмірі өзгергенде браузер жаңа нұсқаны жүктейді.

3.1.1 Тақырып (Header) және навигация

Беттің ең жоғарғы бөлігінде тұрақты тақырып (header) орналасқан. Онда академияның логотипі, навигация мәзірі, тақырып ауыстырғыш батырма және мобильді құрылғыларға арналған «бургер»-мәзір батырмасы бар. Навигация сілтемелері бетті тиісті секцияларға тегіс жылжытады.

```
<header>
  <div class="logo-area">...</div>
  <nav>
    <a href="#home">Главная</a>
    <a href="#curriculum">Учебная Программа</a>
    <a href="#achievements">Достижения</a>
```



```

    <a href="#reviews">Отзйвы</a>
    <a href="#contacts">Контакты</a>
  </nav>
  <button class="theme-toggle" id="themeToggle">...</button>
  <button class="burger">...</button>
</header>

```

Әр навигация сілтемесінің href атрибуты «#» белгісінен басталады — бұл сол id-ге ие секцияға өтуді білдіреді. Мысалы, href="#curriculum" сілтемесі «Оқу бағдарламасы» секциясына апарды. Тақырып бет айналдырылғанда экранда қалып қояды (CSS-те position: sticky), бұл пайдаланушыға кез келген сәтте навигацияны қолдануға мүмкіндік береді.

3.1.2 Басты экран (Hero секциясы)

Басты экран — пайдаланушы көретін бірінші бөлім. Ол академияның атауын, ұранын және «О нас» батырмасын қамтиды. Фонда команданың халықаралық жарыстағы суреті орналасқан.

```

<section id="home" class="gear-bg">
  <div class="section-inner">
    <h1>Smarthub</h1>
    <p class="hero-sub">Первая академия роботехнике и Дронов в Казахстане</p>
    <a href="#about" class="btn btn-white">О нас</a>
  </div>
</section>

```

Мұндағы section тегінің id="home" атрибуты — навигация сілтемесі сол секцияға өту үшін қажет. сілтемесі басылғанда бет «О нас» секциясына тегіс жылжиды. Фондық сурет CSS арқылы қосылады (3.2-бөлімде сипатталады).



3.1-сурет. Басты экранның фондық суреті (WRO 2026 жарысындағы команда)

3.1.3 «Біз туралы» — аударылатын карточкалар

«Біз туралы» секциясында академияның миссиясы екі интерактивтік карточкада көрсетілген. Карточканың алдыңғы жағында қысқаша тақырып, ал артқы жағында толық мәтін орналасқан. Компьютерде тінтуірді апарғанда, ал телефонда карточканы басқанда ол аударылып, артқы жағындағы ақпаратты көрсетеді. Бұл әсер «flip-card» деп аталады.

```
<div class="flip-card">
  <div class="flip-card-inner">
    <div class="flip-card-front">
      <div class="billboard-chip">SmartHub</div>
      <div class="billboard-title">Первая академия робототехники и
дронов</div>
      <p class="billboard-caption">Современное образование...</p>
    </div>
    <div class="flip-card-back">
      <h3>SmartHub</h3>
      <p>SmartHub – первая академия робототехники и дронов...</p>
    </div>
  </div>
</div>
```

Карточканың құрылымы үш қабаттан тұрады: flip-card (сыртқы контейнер, ол тінтуір апаруды немесе басуды «ұстайды»), flip-card-inner

(айналатын ішкі контейнер) және flip-card-front пен flip-card-back (алдыңғы және артқы беттер). Айналу әсері CSS-тің 3D-түрлендірулері арқылы орындалады.

3.1.4 Оқу бағдарламасы

Оқу бағдарламасы секциясы екі бағанадан тұрады: сол жақта Робототехника бағыты деңгейлермен (Junior, Middle, Master, Expert), оң жақта Дрондар, Дрон-футбол және Спорттық PBT бағыттары. Әр деңгей мен бағыт — жоғарыда сипатталған аударылатын карточка. Карточканың алдыңғы жағында деңгейдің атауы мен жасы, артқы жағында толық сипаттама мен оқу мазмұны көрсетіледі.

```
<div class="level-card flip-card">
  <div class="flip-card-inner">
    <div class="flip-card-front">
      <div class="level-icon"><img alt="Junior icon" data-bbox="475 405 490 420"/></div>
      <div class="level-name">Junior</div>
      <div class="level-age">0-2 года</div>
    </div>
    <div class="flip-card-back">
      <h4>Junior</h4>
      <p>Возраст: от 5 лет до 2 класса</p>
      <p>На начальном уровне дети знакомятся с миром робототехники...</p>
    </div>
  </div>
</div>
```

3.1.5 Тулар және жетістіктер галереясы

«Біз қай елдерде болдық» секциясында академия қатысқан елдердің тулары торлы (grid) орналасуда көрсетілген. Тулардың суреттері flagcdn.com сыртқы сервисінен жүктеледі. Жетістігі бар алты елдің туы (Түркия, Оңтүстік Корея, АҚШ, Бельгия, Сингапур, Нидерланды) аударылатын карточка ретінде жасалған: алдыңғы жағында ту мен ел атауы, артқы жағында сол елде жеңіп алынған жетістік сипатталады.

«Жетістіктер» секциясы екі бөліктен тұрады: «Жыл қорытындысы» (сандармен берілген статистика — 6 филиал, 300 оқушы, жүлделі орындар) және фотогалерея. Галерея нақты халықаралық жарыстардан түсірілген суреттерден құралған. Әр сурет бірдей өлшемге келтіріліп (object-fit: cover), тінтуір апарғанда ұлғаяды, ал басқанда толық экранда (лайтбоксте) ашылады.

```

<div class="gallery-grid">
  <figure class="gallery-item">
    
    <figcaption>FIRST Lego League – Champion's Award, Astana 2025</figcaption>
  </figure>
  ... басқа суреттер ...
</div>

```

Мұндағы <figure> және <figcaption> — семантикалық тегтер: сурет пен оның қолжазбасын мағыналық тұрғыдан байланыстырады. loading="lazy" атрибуты «жалқау жүктеу» (lazy loading) деп аталады — сурет тек экранға жақындағанда ғана жүктеледі, бұл беттің жүктелу жылдамдығын арттырады.



3.2-сурет. Фотогалереядағы суреттің мысалы (FIRST Lego League)



3.3-сурет. Fibonacci International Robot Olympiad, Анталья



3.4-сурет. Robotek Grand Tournament Astana — жүлдегерлер



3.5-сурет. Fibonacci EurAsia — Autonomous Car Racing бағытында 2-орын



3.6-сурет. Robotek Grand Tournament — жеңіс кубогы

3.1.6 Пікірлер каруселі

Пікірлер секциясы ата-аналардың пікірлерін көрсететін карусельден тұрады. Барлық пікір карточкалары көлденең жолаққа орналастырылған, оларды сол және оң жақтағы оқтармен немесе төмендегі нүктелермен парақтауға болады. Телефонда саусақпен сырғыту (свайп) арқылы да парақталады.

```
<div class="reviews-carousel">
  <button class="rev-arrow rev-prev"></button>
  <div class="reviews-viewport">
    <div class="reviews-track">
      <div class="review-card">
        <div class="stars">★★★★★</div>
        <p>Мой сын занимается уже 2 года...</p>
        <div class="reviewer">– Айгуль М., мама ученика</div>
      </div>
      ... басқа пікірлер ...
    </div>
  </div>
</div>
```



```

</div>
<button class="rev-arrow rev-next">></button>
<div class="reviews-dots"></div>
</div>

```

3.1.7 Байланыс және өтінім формасы

Байланыс секциясында академияның филиалдары, телефондары және пробалық сабаққа тіркелуге арналған форма орналасқан. Форма пайдаланушыдан атын, телефонын және басқа деректерді сұрайды. Форма жіберілгенде JavaScript оны өңдеп, серверге жібереді.

```

<form id="contactForm">
  <input name="name" placeholder="Ваше имя" />
  <input name="phone" placeholder="Телефон" />
  <input name="age" placeholder="Возраст ребёнка" />
  <input name="direction" placeholder="Направление" />
  <button type="submit">Записаться</button>
</form>

```

Әр <input> өрісінің name атрибуты маңызды — JavaScript нақ осы атаулар арқылы өрістердің мәндерін алады. placeholder атрибуты өріс бос болғанда көрінетін кеңес мәтінін көрсетеді.

3.1.8 Төменгі бөлім (Footer)

Беттің ең төменгі бөлігінде footer (төменгі колонтитул) орналасқан. Онда академия туралы қысқаша мәтін мен авторлық құқық туралы ақпарат болады. footer — семантикалық тег, ол беттің аяқталу бөлігін білдіреді.

```

<footer>
  <p>Первая академия робототехники и дронов в Казахстане
    с современным подходом к образованию.</p>
  <span>&copy; 2026 SmartHub</span>
</footer>

```

Төменгі бөлім бүкіл беттерде бірдей көрінеді және пайдаланушыға академияның негізгі идеясын тағы бір рет еске салады.

3.2 CSS — стильдеу және бейімделгіш дизайн

Сайттың барлық сыртқы көрінісі style.css файлында анықталған. Бұл бөлімде ең маңызды стильдік шешімдер сипатталады.

3.2.1 Аударылатын карточкалардың 3D-әсері

Карточканы аудару әсері CSS-тің 3D-түрлендірулері арқылы жасалады. Негізгі идея: ішкі контейнер (flip-card-inner) Y осі бойынша 180 градусқа

бұрылады, ал алдыңғы мен артқы беттердің «backface-visibility» қасиеті «hidden» етіп қойылады, бұл бұрылған кезде тек бір беттің көрінуін қамтамасыз етеді.

```
.flip-card-inner {
  position: relative;
  transform-style: preserve-3d;
  transition: transform 0.8s cubic-bezier(0.19, 1, 0.22, 1);
}
/* Тінтуір апарғанда (компьютер) аудару */
@media (hover: hover) {
  .flip-card:hover .flip-card-inner { transform: rotateY(180deg); }
}
/* Басқанда (телефон) аудару */
.flip-card.flipped .flip-card-inner { transform: rotateY(180deg); }

.flip-card-front, .flip-card-back {
  position: absolute; inset: 0;
  backface-visibility: hidden;
}
.flip-card-back { transform: rotateY(180deg); }
```

Кодтың жұмыс істеу принципі: transform-style: preserve-3d қасиеті ішкі элементтердің 3D-кеңістікте орналасуын сақтайды. @media (hover: hover) медиа-сұрауы тек тінтуірі бар құрылғыларда (компьютерлерде) ғана жұмыс істейді — сондықтан тінтуір апару әсері телефондарда қосылмайды. Телефондар үшін JavaScript карточкаға «flipped» класын қосады, ол да дәл сол 180 градусқа бұру әсерін береді. Артқы беттің бастапқыда 180 градусқа бұрылып тұруы — ол алдыңғы беттің артында «теріс» тұрып, аударылғанда дұрыс көрінуі үшін қажет.

3.2.2 Күндізгі және түнгі тақырып

Сайтта тақырыпты ауыстыру мүмкіндігі бар. Бұл CSS айнымалыларын қайта анықтау арқылы іске асырылған. body элементіне «dark» класы қосылғанда барлық айнымалылар түнгі мәндерге ауысады, ал олар сайттың барлық жерінде қолданылғандықтан, бүкіл бет бірден қараңғыланады.

```
body.dark {
  --dark: #e8e8e8;
  --light-bg: #181818;
  --white: #242424;
  --card-bg: #2e2e2e;
  --nav-bg: #1e1e1e;
  --blue: #8ea2ff;
}
```

Бұл — CSS айнымалыларының ең күшті артықшылығы: бір ғана «dark» класын қосу/алу арқылы ондаған элементтің түсін бір мезетте өзгертуге болады. Айнымалылардың түнгі мәндері қараңғы фон мен ашық мәтінге сәйкес таңдалған, бұл түнде көзге жайлы болуын қамтамасыз етеді.

3.2.3 Бейімделгіш дизайн (адаптивтілік)

Сайт әртүрлі экран өлшемдеріне бейімделеді. Бұл медиа-сұраулар (media queries) арқылы жасалады — экранның ені белгілі бір мәннен кіші болғанда, элементтердің орналасуы өзгереді. Мысалы, компьютерде үш бағанада тұрған карточкалар планшетте екі, ал телефонда бір бағанаға ауысады.

```
@media (max-width: 900px) {  
  .curriculum-grid { grid-template-columns: 1fr; }  
  .reviews-carousel .review-card { flex-basis: calc((100% - 22px) / 2); }  
}  
@media (max-width: 640px) {  
  .reviews-carousel .review-card { flex-basis: 100%; }  
}
```

Мұндағы max-width: 900px шарты «экран ені 900 пиксельден кіші болса» дегенді білдіреді. Grid және Flexbox орналасу жүйелері осы шарттарда қайта реттеледі, нәтижесінде сайт кез келген құрылғыда оқуға ыңғайлы болып қалады.

3.2.4 Фотогалерея стильдері

Фотогалереядағы суреттер бірдей өлшемді плиткаларға келтірілген. Бұл object-fit: cover қасиеті арқылы жасалады — әртүрлі өлшемдегі суреттер плитканың пропорциясына сай қиылып, бірдей көрінеді. Тінтуір апарғанда сурет жайлап ұлғаяды (scale).

```
.gallery-item {  
  aspect-ratio: 4 / 3;  
  border-radius: 18px;  
  overflow: hidden;  
}  
.gallery-item img {  
  width: 100%; height: 100%;  
  object-fit: cover;  
  transition: transform .5s;  
}  
.gallery-item:hover img { transform: scale(1.07); }
```

3.2.5 Басты экранның фондық суреті

Басты экранның фонында академия командасының суреті орналасқан. Сурет тікелей қойылса, оның үстіндегі мәтін оқылмай қалуы мүмкін, сондықтан суреттің үстіне жартылай мөлдір қара қабат (градиент) салынады. Бұл фотоны көрсете отырып, мәтіннің оқылуын қамтамасыз етеді.

```
#home {
  background-image:
    radial-gradient(ellipse 70% 60% at center,
      rgba(8,8,14,0.62) 0%, rgba(8,8,14,0.3) 100%),
    linear-gradient(rgba(10,10,16,0.55), rgba(10,10,16,0.72)),
    url(hero-bg.jpeg);
  background-size: cover;
  background-position: center;
}
```

Мұндағы background-image қасиетіне үш қабат берілген (олар үстінен-үстіне жазылады): радиалды градиент центрді қараңғылайды, сызықтық градиент бүкіл суретті бір қалыпты күңгірттейді, ал ең астында нақты фото тұрады. background-size: cover суреттің бүкіл секцияны толтыруын, ал center оны ортаға туралауды қамтамасыз етеді. Нәтижесінде ақ түсті тақырып пен батырма фотоның үстінде анық көрінеді.

3.3 JavaScript — интерактивтік логика

Сайттың барлық динамикалық мінез-құлқы script.js файлында жазылған. Бұл бөлімде негізгі функциялардың коды мен жұмыс істеу принципі сипатталады.

3.3.1 Тақырыпты ауыстыру

Тақырып ауыстырғыш батырманы басқанда body элементіне «dark» класы қосылады немесе алынады, ал таңдау localStorage-та сақталады, сондықтан бетті қайта ашқанда таңдалған тақырып есте сақталады.

```
const themeToggle = document.getElementById('themeToggle');
if (localStorage.getItem('theme') === 'dark') document.body.classList.add('dark');
themeToggle.addEventListener('click', () => {
  const isDark = document.body.classList.toggle('dark');
  localStorage.setItem('theme', isDark ? 'dark' : 'light');
});
```

Кодтың жұмысы: document.getElementById арқылы батырма табылады. addEventListener('click', ...) батырмаға басу оқиғасын «тыңдайды». Басылған сайын classList.toggle('dark') «dark» класын қосады немесе алады және оның

жаңа күйін (бар/жоқ) қайтарады. Бұл күй localStorage.setItem арқылы браузердің тұрақты жадына жазылады. Бет жүктелгенде localStorage.getItem алдыңғы таңдауды тексеріп, қажет болса «dark» класын бірден қосады.

3.3.2 Аударылатын карточкаларды басу (телефон)

Жоғарыда айтылғандай, компьютерде карточкалар тінтуірмен аударылады (CSS арқылы), ал телефонда оларды басу қажет. Мұны қамтамасыз ету үшін JavaScript тек сенсорлық құрылғыларда ғана карточкаларға басу оқиғасын қосады.

```
const isTouch = window.matchMedia('(hover: none)').matches;
if (isTouch) {
  document.querySelectorAll('.flip-card').forEach(card => {
    card.addEventListener('click', () => card.classList.toggle('flipped'));
  });
}
```

window.matchMedia('(hover: none)') — құрылғыда тінтуір жоқ па (яғни ол сенсорлық па) екенін тексереді. Егер солай болса, барлық flip-card элементтері табылып (querySelectorAll), әрқайсысына басу оқиғасы тағайындалады. Басқанда «flipped» класы ауысып, карточка аударылады. Бұл шешімнің артықшылығы — компьютерде тінтуір мен басудың қақтығысы болмайды.

3.3.3 Пікірлер каруселі

Карусель оқтармен парақтауды, нүкте-индикаторларды және саусақпен сырғытуды қолдайды. Ол браузердің табиғи көлденең айналдыруына (scroll-snap) негізделген, сондықтан телефонда свайп автоматты түрде жұмыс істейді.

```
const track = carousel.querySelector('.reviews-track');
const cardStep = () => cards[0].offsetWidth + gap;
const perView = () => Math.round((track.clientWidth + gap) / cardStep());

function scrollToPage(i) {
  track.scrollTo({ left: i * cardStep() * perView(), behavior: 'smooth' });
}
prev.addEventListener('click', () => scrollToPage(currentPage() - 1));
next.addEventListener('click', () => scrollToPage(currentPage() + 1));
```

Карусельдің логикасы: cardStep() бір карточканың енін (аралықпен бірге) есептейді, perView() экранға қанша карточка сыятынын анықтайды. Оқ басылғанда scrollToPage функциясы жолақты (track) бір «бет» шамасына тегіс (behavior: smooth) айналдырады. Нүктелер ағымдағы бетті көрсетеді және браузердің айналдыру оқиғасында жаңартылып отырады.

3.3.4 Фото лайтбоксы

Галереядағы суретті басқанда ол толық экранда ашылады. Лайтбокс — бетке динамикалық түрде қосылатын қабат (overlay). Онда сурет, оның қолжазбасы, парақтау оқтары және жабу батырмасы болады. Esc пернесімен немесе фонды басқанда ол жабылады.

```
items.forEach((fig, i) => {
  fig.addEventListener('click', () => open(i));
});
function show(i) {
  current = (i + slides.length) % slides.length;
  imgEl.src = slides[current].src;
  capEl.textContent = slides[current].caption;
}
document.addEventListener('keydown', (e) => {
  if (e.key === 'Escape') close();
  else if (e.key === 'ArrowLeft') show(current - 1);
  else if (e.key === 'ArrowRight') show(current + 1);
});
```

Мұндағы show(i) функциясы көрсетілетін суретті ауыстырады. (i + slides.length) % slides.length өрнегі — циклдік парақтауды қамтамасыз етеді: соңғы суреттен кейін қайтадан біріншісіне оралады. keydown оқиғасы пернетақтаны тыңдайды: Escape — жабу, көрсеткі пернелер — алдыңғы/келесі суретке өту.

3.3.5 Өтінім формасын өңдеу

Бұл — клиенттік бөліктің ең маңызды функциясы. Пайдаланушы форманы жібергенде, JavaScript келесі әрекеттерді орындайды: деректерді жинайды, тексереді, алдымен браузердің жадына (localStorage) сақтайды, содан кейін серверге жібереді. Деректерді алдын ала жергілікті сақтаудың себебі — сервер қолжетімсіз болса да, пайдаланушының өтінімі жоғалмауы тиіс.

```
document.getElementById('contactForm').addEventListener('submit', async function(e)
{
  e.preventDefault(); // беттің қайта жүктелуін болдырмай
  const data = {
    name:      this.querySelector('[name="name"]').value.trim(),
    phone:     this.querySelector('[name="phone"]').value.trim(),
    age:       this.querySelector('[name="age"]').value.trim(),
    direction: this.querySelector('[name="direction"]').value.trim(),
  };
  if (!data.name) { toast('Укажите ваше имя', false); return; }
  if (!data.phone) { toast('Укажите номер телефона', false); return; }

  saveToLocal({ ...data }); // жергілікті сақтау
  toast(`Спасибо, ${data.name}!`, true); // хабарлама көрсету
  this.reset(); // форманы тазалау
```

```

fetch('/api/submit', { // серверге фонда жіберу
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify(data),
}).catch(() => {});
});

```

Кодтың жұмысы қадам-қадаммен: `e.preventDefault()` форма жіберілгенде беттің әдеттегідей қайта жүктелуін тоқтатады. Содан кейін форманың әр өрісінен мән алынып (`querySelector + .value`), артық бос орындар `.trim()` арқылы алынады. Егер аты немесе телефоны бос болса, `toast` функциясы қызыл хабарлама көрсетеді әрі әрі қарай жалғаспайды (`return`). Деректер дұрыс болса, `saveToLocal` оларды `localStorage`-қа жазады, пайдаланушыға алғыс хабарламасы көрсетіледі, форма тазаланады. Соңында `fetch` функциясы деректерді JSON форматында серверге POST әдісімен жібереді. `.catch(() => {})` — сервер қолжетімсіз болса да қате шықпай, өтінімнің жергілікті жадта сақталып қалуын қамтамасыз етеді.

Деректерді жергілікті сақтайтын `saveToLocal` функциясы келесідей:

```

function saveToLocal(data) {
  const list = JSON.parse(localStorage.getItem('sh_submissions') || '[]');
  list.unshift({
    ...data,
    id: Date.now(),
    created_at: new Date().toLocaleString('ru-RU'),
  });
  localStorage.setItem('sh_submissions', JSON.stringify(list));
}

```

Бұл функция `localStorage`-тан бұрынғы өтінімдер тізімін оқиды (немесе бос тізім жасайды), оған жаңа өтінімді (уақыт белгісі мен бірегей `id`-мен бірге) тізімнің басына қосады (`unshift`) және қайтадан сақтайды. `JSON.parse` пен `JSON.stringify` — мәтін мен объект арасында түрлендіру функциялары, өйткені `localStorage` тек мәтінді сақтай алады.

3.3.6 Навигацияның белсенді сілтемесі

Пайдаланушы бетті айналдырғанда, навигациядағы ағымдағы секцияға сәйкес сілтеме автоматты түрде бөлектеледі. Бұл `IntersectionObserver` браузер интерфейсі арқылы іске асырылады — ол қай секцияның экранда екенін бақылайды.

```
const observer = new IntersectionObserver(entries => {
  entries.forEach(entry => {
    if (entry.isIntersecting) {
      navLinks.forEach(a => {
        a.classList.remove('active');
        if (a.getAttribute('href') === '#' + entry.target.id)
          a.classList.add('active');
      });
    }
  });
}, { threshold: 0.35 });
sections.forEach(s => observer.observe(s));
```

IntersectionObserver — экранда қандай элементтердің көрінетінін тиімді бақылайтын құрал. Секцияның 35%-ы экранға кіргенде (threshold: 0.35), оған сәйкес навигация сілтемесіне «active» класы қосылып, қалғандарынан алынады. Бұл пайдаланушыға беттің қай бөлімінде тұрғанын көрсетеді.

3.3.7 Мобильді мәзір (бургер)

Кіші экрандарда навигация мәзірі жасырылып, оның орнына үш сызықты «бургер»-батырма көрсетіледі. Оны басқанда мәзір ашылады немесе жабылады.

```
const burger = document.querySelector('.burger');
const nav = document.querySelector('nav');
burger.addEventListener('click', () => nav.classList.toggle('open'));
nav.querySelectorAll('a').forEach(a =>
  a.addEventListener('click', () => nav.classList.remove('open')));
```

Бургер-батырма басылғанда nav элементіне «open» класы қосылады/алынады, ол CSS арқылы мәзірді көрсетеді немесе жасырады. Сондай-ақ мәзірдегі кез келген сілтеме басылғанда мәзір автоматты түрде жабылады — бұл мобильді пайдаланушыға ыңғайлы.

3.3.8 Хабарлама терезесі (toast)

Форма жіберілгенде немесе қате болғанда пайдаланушыға қалқымалы хабарлама (toast) көрсетіледі. Ол беттің төменгі жағында пайда болып, бірнеше секундтан кейін өздігінен жоғалады.

```
function toast(msg, ok) {
  let el = document.getElementById('formToast');
  if (!el) { el = document.createElement('div'); el.id = 'formToast';
    document.body.appendChild(el); }
  el.textContent = msg;
  el.style.background = ok ? '#1e8c3a' : '#CC0000';
  el.style.opacity = '1';
  clearTimeout(el._t);
  el._t = setTimeout(() => el.style.opacity = '0', 4500);
}
```


Хабарлама сәтті болса жасыл (#1e8c3a), қате болса қызыл (#CC0000) түспен көрсетіледі. `setTimeout` функциясы 4500 миллисекундтан (4,5 секунд) кейін хабарламаны мөлдірлендіреді (жасырады). `clearTimeout` алдыңғы таймерді тоқтатады, бұл хабарламалар тез ауысқанда қателікті болдырмайды.

4. СЕРВЕРЛІК БӨЛІК

4.1 Node.js және Express негіздері

Серверлік бөлік server.js файлында орналасқан және Node.js ортасында Express фреймворкі негізінде жұмыс істейді. Сервердің негізгі міндеттері: статикалық файлдарды (HTML, CSS, JS, суреттер) браузерге беру, өтінімдерді қабылдап сақтау, әкімшіге деректерді көрсету және Excel экспортын жасау.

Сервердің басында қажетті модульдер қосылып, негізгі параметрлер анықталады:

```
const express = require('express');
const XLSX = require('xlsx');
const fs = require('fs');
const path = require('path');

const app = express();
const PORT = 3000;
const ADMIN_KEY = 'smarthub2025';
const DB_FILE = path.join(__dirname, 'submissions.json');
```

require() функциясы арқылы үш модуль қосылады: express — веб-сервер үшін, xlsx — Excel файлдарын жасау үшін, fs (file system) — файлдармен жұмыс істеу үшін, path — файл жолдарын дұрыс құрастыру үшін. PORT — сервер тыңдайтын порт нөмірі (3000), ADMIN_KEY — әкімшілік панельге кіру кілті, DB_FILE — деректер базасы файлының толық жолы.

Express-те «middleware» (аралық өңдеуші) деген ұғым бар — әр сұраныс өткізілетін функциялар. Бұл жобада екі middleware қолданылады:

```
app.use(express.json()); // JSON денесін оқу
app.use(express.static(path.join(__dirname))); // статикалық файлдарды беру
```

express.json() — клиенттен келген JSON деректерін автоматты түрде объектіге түрлендіреді. express.static() — жоба қалтасындағы барлық файлдарды (index.html, style.css, суреттер) браузерге беруге мүмкіндік береді. Осы екі жолдың арқасында пайдаланушы localhost:3000 мекенжайын ашқанда index.html беті көрінеді.

Бұл жобада сервер HTTP протоколының үш әдісін қолданады. Олардың әрқайсысы белгілі бір әрекетке жауап береді (4.1-кесте).

4.1-кесте. Қолданылған HTTP әдістері

HTTP әдісі	Мақсаты	Жобадағы қолданысы
GET	Деректерді алу	Өтінімдер тізімін алу, Excel экспорты
POST	Жаңа дерек жіберу	Жаңа өтінімді сақтау
DELETE	Деректі жою	Өтінімді жою

Сервер клиентке жауап бергенде «статус коды» деп аталатын нөмірді де жібереді. Ол сұраныстың сәтті немесе сәтсіз болғанын білдіреді (4.2-кесте).

4.2-кесте. Қолданылған HTTP статус кодтары

Код	Мағынасы	Жобадағы жағдай
200	Сәтті орындалды	Өтінім сақталды немесе деректер қайтарылды
400	Қате сұраныс	Аты немесе телефоны толтырылмаған
401	Рұқсат жоқ	Әкімшілік кілт қате

4.2 Деректер базасы (submissions.json)

Бұл жобада толыққанды деректер базасын басқару жүйесінің (мысалы, MySQL немесе PostgreSQL) орнына жеңіл әрі қарапайым файлдық JSON-база қолданылған. Барлық өтінімдер submissions.json файлында сақталады. Бұл шешім шағын жоба үшін орынды: ол қосымша бағдарлама орнатуды қажет етпейді, оңай оқылады және сақтық көшірмесін жасау жеңіл.

Базаның құрылымы — nextId (келесі өтінімнің реттік нөмірі) және rows (өтінімдер массиві) өрістерінен тұрады:

```
{
  "nextId": 2,
  "rows": [
    {
      "id": 1,
      "name": "Асан",
      "phone": "+7 777 123 45 67",
      "city": "Астана",
    }
  ]
}
```

```

    "country": "Казахстан",
    "message": "Хочу записаться",
    "created_at": "22.05.2026, 03:19:44"
  }
]
}

```

Базамен жұмыс істеу үшін екі көмекші функция жазылған: loadDB (базаны оқу) және saveDB (базаны сақтау):

```

function loadDB() {
  if (!fs.existsSync(DB_FILE)) return { nextId: 1, rows: [] };
  try { return JSON.parse(fs.readFileSync(DB_FILE, 'utf8')); }
  catch { return { nextId: 1, rows: [] }; }
}
function saveDB(db) {
  fs.writeFileSync(DB_FILE, JSON.stringify(db, null, 2), 'utf8');
}

```

loadDB функциясы алдымен файлдың бар-жоғын тексереді (existsSync). Егер файл жоқ болса немесе оқу кезінде қате шықса, бос база қайтарылады. Файл бар болса, оның мазмұны оқылып (readFileSync), JSON.parse арқылы объектіге түрлендіріледі. saveDB функциясы керісінше: объектіні JSON мәтініне айналдырып (JSON.stringify, мұндағы «2» — кодты әдемі шегініспен жазу үшін), файлға жазады (writeFileSync).

4.3 API эндпоинттері

Сервер клиентпен «API эндпоинттері» (бағыттар) арқылы байланысады. Әр эндпоинт белгілі бір URL-ге келген сұранысты өңдейді. Бұл жобада төрт эндпоинт бар.

4.3.1 Өтінімді сақтау (POST /api/submit)

Бұл эндпоинт клиенттен келген өтінімді қабылдап, базаға сақтайды:

```

app.post('/api/submit', (req, res) => {
  const { name, phone, city, country, message } = req.body;
  if (!name || !name.trim()) return res.status(400).json({ error: 'Укажите имя' });
  if (!phone || !phone.trim()) return res.status(400).json({ error: 'Укажите телефон' });

  const db = loadDB();
  const record = {
    id: db.nextId++,
    name: name.trim(), phone: phone.trim(),
    city: (city || '').trim(), country: (country || '').trim(),
    message: (message || '').trim(),
    created_at: new Date().toLocaleString('ru-RU', { timeZone: 'Asia/Almaty' }),
  };
  db.rows.unshift(record);
  saveDB(db);
}

```

```
res.json({ ok: true, id: record.id });
});
```

`app.post('/api/submit', ...)` — «/api/submit» мекенжайына POST әдісімен келген сұранысты өңдейді. `req.body` — клиент жіберген деректер. Алдымен аты мен телефонның бос еместігі тексеріледі; бос болса, 400 (қате) статусымен жауап қайтарылады. Деректер дұрыс болса, жаңа жазба (`record`) құрылады: оған бірегей `id` беріледі (`db.nextId++` — нөмірді алып, бірге арттырады), уақыт белгісі қойылады (Алматы уақыт белдеуінде). Жазба базаның басына қосылып (`unshift`), база сақталады. Соңында клиентке сәтті орындалғаны туралы `{ ok: true }` жауабы жіберіледі.

4.3.2 Әкімшілік қорғаныс (middleware)

Әкімшілікке арналған эндпоинттер кілтпен қорғалған. Бұл `requireAdmin` аралық функциясы арқылы жасалады:

```
function requireAdmin(req, res, next) {
  const key = req.query.key || req.headers['x-admin-key'];
  if (key !== ADMIN_KEY) return res.status(401).json({ error: 'Неверный ключ доступа' });
  next();
}
```

Бұл функция сұраныстағы кілтті (URL параметрінен немесе тақырыптан) тексереді. Егер кілт дұрыс болмаса, 401 (рұқсат жоқ) статусы қайтарылады. Кілт дұрыс болса, `next()` шақырылып, сұраныс әрі қарай өңделеді. Осы функция әкімшілік эндпоинттерге қосылады, сондықтан кілтсіз ешкім өтінімдерді көре алмайды.

4.3.3 Өтінімдерді көру және жою

```
// Барлық өтінімдерді алу
app.get('/api/submissions', requireAdmin, (req, res) => {
  res.json(loadDB().rows);
});
// Өтінімді жою
app.delete('/api/submissions/:id', requireAdmin, (req, res) => {
  const db = loadDB();
  const id = Number(req.params.id);
  db.rows = db.rows.filter(r => r.id !== id);
  saveDB(db);
  res.json({ ok: true });
});
```

Бірінші эндпоинт (GET) барлық өтінімдерді қайтарады — `requireAdmin` қорғанысы оған дейін орындалады. Екінші эндпоинт (DELETE) белгілі бір `id`-

мен өтінімді жояды: req.params.id арқылы URL-дегі нөмір алынады, filter функциясы сол id-ден басқа барлық жазбаларды қалдырады (яғни керектісін алып тастайды), база сақталады.

4.4 Excel форматына экспорт

Әкімші барлық өтінімдерді бір рет басу арқылы Excel файлына түсіріп ала алады. Бұл функция.xlsx кітапханасы арқылы орындалады:

```
app.get('/api/export', requireAdmin, (req, res) => {
  const { rows } = loadDB();
  const data = rows.map(r => ({
    'ID': r.id, 'Имя': r.name, 'Телефон': r.phone,
    'Город': r.city, 'Страна': r.country,
    'Сообщение': r.message, 'Дата': r.created_at,
  }));
  const ws = XLSX.utils.json_to_sheet(data);
  const wb = XLSX.utils.book_new();
  XLSX.utils.book_append_sheet(wb, ws, 'Заявки SmartHub');
  const buffer = XLSX.write(wb, { type: 'buffer', bookType: 'xlsx' });
  res.setHeader('Content-Disposition', 'attachment; filename=smarthub-zayavki.xlsx');
  res.send(buffer);
});
```

Кодтың жұмысы: алдымен барлық өтінімдер базадан оқылып, map функциясы арқылы Excel бағандарына сай (ID, Имя, Телефон, ...) объектілерге түрлендіріледі. XLSX.utils.json_to_sheet деректерді кесте-параққа (worksheet) айналдырады, book_new жаңа кітап жасайды, book_append_sheet парақты кітапқа қосады. XLSX.write кітапты екілік буферге (файл деректеріне) жазады. Content-Disposition тақырыбы браузерге файлды жүктеп алу керектігін және оның атын (smarthub-zayavki.xlsx) хабарлайды. Соңында файл клиентке жіберіледі.

4.5 Серверді іске қосу

Барлық эндпоинттер анықталғаннан кейін сервер белгілі бір портты «тыңдай» бастайды. Бұл app.listen функциясы арқылы жасалады:

```
app.listen(PORT, () => {
  console.log(`Сервер запущен: http://localhost:${PORT}`);
  console.log(`Админка: http://localhost:${PORT}/admin.html`);
  console.log(`Ключ доступа: ${ADMIN_KEY}`);
});
```

app.listen(PORT, ...) сервердің 3000-портта жұмыс істей бастауын білдіреді. Сервер сәтті іске қосылғанда console.log арқылы консольге сайттың,

әкімшілік панельдің мекенжайлары мен кіру кілті шығады. Осыдан кейін браузерден localhost:3000 мекенжайын ашуға болады.

5. АДМИНИСТРАТОРЛЫҚ ПАНЕЛЬ

Әкімшілік панель — академия қызметкеріне арналған жеке бет (admin.html). Ол арқылы барлық түскен өтінімдерді көруге, жоюға және Excel-ге экспорттауға болады. Панель сайттан бөлек, жеке мекенжайда (localhost:3000/admin.html) орналасқан.

5.1 Аутентификация (кіру кілті)

Панельге кіру кілтпен қорғалған. Бет ашылғанда алдымен кіру терезесі (login overlay) көрінеді. Қызметкер дұрыс кілтті (smarthub2025) енгізгеннен кейін ғана панельдің мазмұны ашылады. Кілт серверге әр сұранысқа жіберіліп, тексеріледі. Бұл — деректерге рұқсатсыз қол жеткізуді болдырмаудың қарапайым әрі тиімді тәсілі.

Кіру терезесінің логикасы JavaScript-те жазылған. Қызметкер енгізген кілт серверге тексеру үшін жіберіледі — егер сервер 200 статусымен жауап берсе, кілт дұрыс деп саналып, панель ашылады:

```
async function login(key) {
  const res = await fetch('/api/submissions?key=' + key);
  if (res.ok) {
    adminKey = key;
    loginOverlay.classList.add('hidden'); // кіру терезесін жасыру
    loadSubmissions();                    // өтінімдерді жүктеу
  } else {
    loginError.textContent = 'Неверный ключ доступа';
  }
}
```

fetch функциясы серверге кілтпен сұраныс жібереді. res.ok — жауаптың сәтті (200) екенін білдіреді. Сәтті болса, кілт есте сақталып, кіру терезесі жасырылады және өтінімдер жүктеледі. Қате болса, пайдаланушыға қызыл ескерту көрсетіледі.

5.2 Заявкаларды басқару

Өтінімдерді кесте түрінде көрсету де JavaScript арқылы динамикалық орындалады. Сервер қайтарған деректер цикл арқылы кестенің жолдарына түрлендіріледі:

```
function renderTable(rows) {
  tbody.innerHTML = rows.map(r => `
```



```

    <tr>
      <td>${r.id}</td>
      <td>${r.name}</td>
      <td>${r.phone}</td>
      <td>${r.city}</td>
      <td>${r.created_at}</td>
      <td><button onclick="del(${r.id})">Удалить</button></td>
    </tr>`).join('');
  }

```

Бұл функция әр өтінімді (r) HTML кесте жолына (<tr>) айналдырады. map функциясы барлық жолдарды жасап, join оларды біріктіреді, ал innerHTML кестенің денесіне (tbody) орналастырады. Әр жолдағы «Удалить» батырмасы басылғанда del(id) функциясы шақырылып, сол өтінімді сервердегі DELETE эндпоинт арқылы жояды.

Кіргеннен кейін қызметкер барлық өтінімдерді кесте түрінде көреді. Әр жолда өтінім берушінің аты, телефоны, қаласы, елі, хабарламасы және түскен уақыты көрсетіледі. Әр өтінімді жеке жоюға болады, сондай-ақ «Excel-ге экспорттау» батырмасы барлық деректерді бір файлға түсіреді. Деректер сервердегі /api/submissions және /api/export эндпоинттерінен алынады. Осылайша әкімшілік панель серверлік API арқылы деректер базасымен байланысады.

6. ҚАУІПСІЗДІК, ТЕСТІЛЕУ ЖӘНЕ ОРНАЛАСТЫРУ

6.1 Қауіпсіздік шаралары

Жобада бірнеше қауіпсіздік шарасы қарастырылған:

- **Кіру кілті** — әкімшілік эндпоинттер ADMIN_KEY кілтімен қорғалған, кілтсіз деректерге қол жеткізу мүмкін емес;
- **Серверлік тексеру (валидация)** — өтінім деректерінің дұрыстығы тек клиентте ғана емес, серверде де тексеріледі (бос аты/телефоны қабылданбайды);
- **Деректерді тазалау** — барлық кіріс деректер .trim() арқылы артық бос орындардан тазартылады.

Болашақта қауіпсіздікті арттыру үшін кілтті құпия сөзбен (хэшпен) ауыстыру, HTTPS протоколын қосу және сұраныс жиілігін шектеу (rate limiting) ұсынылады.

6.2 Тестілеу

Жоба әзірлеу барысында қолмен (manual) тестіленді. Тексерілген сценарийлер: форманы дұрыс және қате деректермен жіберу, тақырыпты ауыстыру, карточкаларды компьютер мен телефонда аудару, каруселді парақтау, лайтбоксты ашу/жабу, әкімшілік панельге дұрыс және қате кілтпен кіру, өтінімді жою және Excel-ге экспорттау. Барлық сценарийлер күтілген нәтиже берді. Сайттың бейімделгіштігі браузердің әзірлеуші құралдары арқылы әртүрлі экран өлшемдерінде тексерілді.

6.3 Жобаны іске қосу және орналастыру

Жобаны жергілікті компьютерде іске қосу үшін келесі қадамдар орындалады:

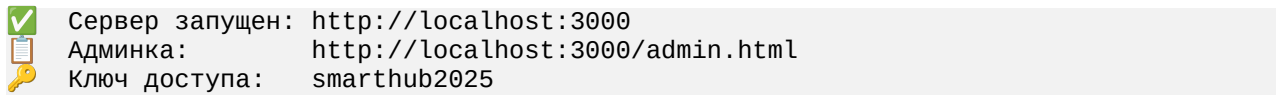
13. Node.js орнату;

14. жоба қалтасында «npm install» командасымен тәуелділіктерді орнату;

15. «npm start» немесе «node server.js» командасымен серверді іске қосу;

16.браузерде localhost:3000 мекенжайын ашу.

Сервер іске қосылғанда консольде келесі ақпарат шығады:



Сервер запущен: <http://localhost:3000>
Админка: <http://localhost:3000/admin.html>
Ключ доступа: smarthub2025

Жобаны нақты интернетке орналастыру (deploy) үшін Render, Railway немесе VPS сияқты хостингтерді қолдануға болады, өйткені жоба Node.js серверін қажет етеді.

ҚОРЫТЫНДЫ

Бұл дипломдық жобада SmartHub робототехника және дрон академиясы үшін толыққанды веб-сайт әзірленді. Жоба барысында қойылған барлық мақсат-міндеттерге қол жеткізілді.

Жобаның клиенттік бөлігінде HTML5, CSS3 және JavaScript технологиялары қолданыла отырып, заманауи әрі бейімделгіш бір беттік сайт құрылды. Сайтта интерактивтік элементтер — тінтуірмен және тиекпен аударылатын карточкалар, пікірлер каруселі, толық экрандық фотогалерея (лайтбокс) және күндізгі/түнгі тақырыпты ауыстыру — іске асырылды. Барлық элементтер компьютерде, планшетте және смартфонда дұрыс жұмыс істейді.

Жобаның серверлік бөлігінде Node.js және Express технологиялары негізінде сервер құрылды. Ол статикалық файлдарды беруді, өтінімдерді қабылдап деректер базасына (submissions.json) сақтауды және оларды Excel форматына экспорттауды қамтамасыз етеді. Сонымен қатар өтінімдерді қарауға, жоюға және түсіруге арналған, кілтпен қорғалған әкімшілік панель әзірленді.

Жобада деректердің сенімділігіне ерекше көңіл бөлінді: өтінім алдымен браузердің жадына (localStorage) сақталып, содан кейін серверге жіберіледі, бұл сервер қолжетімсіз болған жағдайда да деректің жоғалмауын қамтамасыз етеді.

Әзірленген веб-сайт нақты академияның жұмысында қолданылуға толық дайын. Ол академияның онлайн-беделін арттырады, жаңа клиенттерді тартады және өтінімдерді жинау процесін автоматтандырады. Болашақта жобаны нақты деректер базасына (MySQL, PostgreSQL) көшіру, төлем жүйесін қосу және көп тілді нұсқасын жасау арқылы дамытуға болады.

Осы дипломдық жобаны орындау барысында мен веб-әзірлеудің теориялық және практикалық дағдыларын — клиенттік және серверлік бағдарламалауды, деректермен жұмыс істеуді және жобаны толық циклмен жасауды меңгердім деп ойлаймын.

ПАЙДАЛАНЫЛҒАН ӘДЕБИЕТТЕР ТІЗІМІ

1. Дэвид Флэнаган. JavaScript. Толық нұсқаулық. — М.: Вильямс, 2021.
2. Робин Никсон. HTML5, CSS3 және JavaScript негіздерін үйренеміз. — СПб.: Питер, 2019.
3. MDN Web Docs — веб-технологиялардың ресми құжаттамасы. URL: <https://developer.mozilla.org>
4. Express.js ресми құжаттамасы. URL: <https://expressjs.com>
5. Node.js ресми құжаттамасы. URL: <https://nodejs.org>
6. Итан Маркотт. Адаптивті веб-дизайн (Responsive Web Design). — A Book Apart, 2014.
7. SheetJS (xlsx) кітапханасының құжаттамасы. URL: <https://sheetjs.com>
8. Джон Дакетт. HTML и CSS. Веб-сайттарды әзірлеу және құру. — М.: Эксмо, 2017.
9. W3Schools — веб-технологияларды үйренуге арналған ресурс. URL: <https://www.w3schools.com>
10. Кайл Симпсон. Сіз JavaScript-ті білмейсіз (You Don't Know JS). — O'Reilly, 2015.